

Best Performing Model (Tuned GBM)

```
In [1]: # Import packages
import pandas as pd
import numpy as np

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import SimpleImputer, IterativeImputer
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import cross_validate, cross_val_predict, StratifiedKFold
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder
from sklearn.pipeline import make_pipeline
from sklearn.metrics import precision_recall_curve
from sklearn.ensemble import HistGradientBoostingClassifier
```

```
In [ ]: # Define training and test data filepaths
train_filepath = '../datasets/train.csv'
test_filepath = '../datasets/test.csv'
```

1. Data Preparation and Dealing with Missing Data

```
In [3]: # Load both training data and test data
train_df = pd.read_csv(train_filepath)
test_df = pd.read_csv(test_filepath)
```

Data Preparation

```
In [4]: # Store 'ID' from test data
test_ids = test_df['ID'].copy()

# Drop 'ID' column
train_df = train_df.drop(columns=['ID'])
test_df = test_df.drop(columns=['ID'])
```

```
In [5]: # Define function to preprocess dataframes

def prepare_data(df):
    """
    Clean and preprocess each variable in the dataset by performing transformation
    and mapping. Details on how each variables are cleaned and preprocessed are provided
    via block comments.

    Parameters
    -----
    df: pandas.DataFrame
        Dataset to perform preprocessing on. Must include the following columns:
        'occupation', 'marital_status', 'education', 'default', 'has_mortgage',
        'has_pers_loan', 'contact_type', 'last_contact_date', 'age',
        'avg_balance', 'n_curr_campaign_contacts', 'total_call_time',
        'prev_days', 'n_prev_campaign_contacts', and 'prev_outcome'.

    Returns
    -----
    df: pandas.DataFrame
        Cleaned and preprocessed dataframe via the process explained in the block comments.
    """
```

Notes

For 'last_contact_date', different datasets might have different format. The default format is 'YYYY-MM-DD', however issues might be encountered if other date formats are specified, and the function `pd.to_datetime()` assumes the wrong date format. An exception test is added to check if the converted dates are not in the appropriate ranges. A warning message is expected if the dataset applied to this function have a different date format other than 'YYYY-MM-DD'.

"""

----CATEGORICAL VARIABLES----

---occupation---

Remove spaces in occupation labels

```
df['occupation'] = df['occupation'].str.strip()
```

Map occupation categories

```
occupation_dict = {
    'student': 'student',
    'retired': 'retired',
    'unemployed': 'unemployed',
    'management': 'management',
    'self-employed': 'self-employed',
    'unknown': 'unknown',
    'admin.': 'admin',
    'technician': 'technician',
    'services': 'other',
    'housemaid': 'other',
    'entrepreneur': 'other',
    'blue-collar': 'other',
    np.nan: 'unknown'
}
```

```
def map_occupation(value):
    return occupation_dict[value]
```

Apply mapping to occupation

```
df['occupation'] = df['occupation'].apply(map_occupation)
```

---marital_status---

Remove spaces and lowercase in marital_status labels

```
df['marital_status'] = df['marital_status'].str.strip().str.lower()
```

Map marital_status categories and add missing category for na data

```
marital_status_dict = {
    'married': 'married',
    'single': 'single',
    'divorced': 'divorced',
    'div': 'divorced',
    np.nan: 'missing'
}
```

```
def map_marital_status(value):
    return marital_status_dict[value]
```

Apply mapping to marital_status

```
df['marital_status'] = df['marital_status'].apply(map_marital_status)
```

---education---

Remove spaces for education labels

```
df['education'] = df['education'].str.strip()
```

```

# Convert education to a numerical variable, and let 'unknown' be missing values
education_dict = {
    'primary': 1,
    'secondary': 2,
    'tertiary': 3,
    'unknown': np.nan,
    np.nan: np.nan
}

def map_education(value):
    return education_dict[value]

# Apply mapping to education
df['education'] = df['education'].apply(map_education)

# ---default---

# Convert default to a numerical variable
default_dict = {
    'no': 0,
    'yes': 1,
    np.nan: np.nan
}

def map_default(value):
    return default_dict[value]

# Apply mapping to default
df['default'] = df['default'].apply(map_default)

# ---has_mortgage---

# Convert has_mortgage to a numerical variable
has_mortgage_dict = {
    'no': 0,
    'yes': 1,
    np.nan: np.nan
}

def map_has_mortgage(value):
    return has_mortgage_dict[value]

# Apply mapping to has_mortgage
df['has_mortgage'] = df['has_mortgage'].apply(map_has_mortgage)

# ---has_pers_loan---

# Convert has_pers_loan to a numerical variable
has_pers_loan_dict = {
    'no': 0,
    'yes': 1,
    np.nan: np.nan
}

def map_has_pers_loan(value):
    return has_pers_loan_dict[value]

# Apply mapping to has_pers_loan
df['has_pers_loan'] = df['has_pers_loan'].apply(map_has_pers_loan)

# ---contact_type---

# Map contact_type categories and let na values be unknown
contact_type_dict = {
    'cellular': 'cellular',
    'telephone': 'telephone',
    'unknown': 'unknown',

```

```

np.nan: 'unknown'
}
def map_contact_type(value):
    return contact_type_dict[value]

# Apply mapping to contact_type
df['contact_type'] = df['contact_type'].apply(map_contact_type)

# ---last_contact_date---

# Convert last_contact_date to datetime with YYYY-MM-DD
df['last_contact_date'] = pd.to_datetime(df['last_contact_date'],
                                         errors='coerce')

# Create new variables for month, day, and weekday
# No need for year, since all last_contact_date is in year 2024
df['last_contact_month'] = df['last_contact_date'].dt.month
df['last_contact_day'] = df['last_contact_date'].dt.day
df['last_contact_weekday'] = df['last_contact_date'].dt.weekday

# Exception check for month, day and weekday
invalid_months = df.loc[df['last_contact_month'].notna() \
                        & ((df['last_contact_month'] < 1) | \
                          (df['last_contact_month'] > 12))]
invalid_days = df.loc[df['last_contact_day'].notna() \
                     & ((df['last_contact_day'] < 1) | \
                       (df['last_contact_day'] > 31))]
invalid_weekdays = df.loc[df['last_contact_weekday'].notna() \
                          & ((df['last_contact_weekday'] < 0) | \
                            (df['last_contact_weekday'] > 6))]

if not invalid_months.empty or not invalid_days.empty or not invalid_weekdays.empty:
    raise Exception("Wrong datetime format specification.")

# drop original last_contact_date column
df = df.drop(columns=['last_contact_date'])

##### -----NUMERICAL VARIABLES----- #####

# ---age---

# Replace age wrongly specified as year of birth with 2025 - age
df.loc[df['age'] > 500, 'age'] = 2025 - df.loc[df['age'] > 500, 'age']

# Replace age below 18 (-1 and 0) with NaN
df.loc[df['age'] < 18, 'age'] = np.nan

# ---avg_balance---

# convert avg_balance to numeric variable
df['avg_balance'] = pd.to_numeric(df['avg_balance'], errors='coerce')

# ---n_curr_campaign_contacts---

# Replace -1 values with NaN
df.loc[df['n_curr_campaign_contacts'] == -1, 'n_curr_campaign_contacts'] = np.nan

# ---total_call_time---

# Replace -1 values with NaN
df.loc[df['total_call_time'] == -1, 'total_call_time'] = np.nan

```

```
##### -----group prev_days, n_prev_campaign_contacts and prev_outcome----- #####

# 1. Fix prev_days and n_prev_campaign_contacts inconsistent use of delimiters

# Identify rows with semi-colon
semicolon_rows = df['prev_days'].astype(str).str.contains(';', na=False)

# Split prev_days
split_prev_days = df.loc[semicolon_rows, 'prev_days']\
                    .astype(str).str.split(';', expand=True)

# Assign first values back to prev_days
df.loc[semicolon_rows, 'prev_days'] = split_prev_days[0].astype(float)

# Assign second values to n_prev_campaign_contacts
df.loc[semicolon_rows, 'n_prev_campaign_contacts'] = split_prev_days[1].astype(float)

# Convert both columns to float (to ensure consistency)
df['prev_days'] = df['prev_days'].astype(float)
df['n_prev_campaign_contacts'] = df['n_prev_campaign_contacts'].astype(float)

# 2. Replace -1 and 0 with NaN in prev_days and n_prev_campaign_contacts
df[['prev_days']] = df[['prev_days']].replace([-1, 0], np.nan)
df[['n_prev_campaign_contacts']] = df[['n_prev_campaign_contacts']]\
                                   .replace([-1, 0], np.nan)

# 3. For prev_days, n_prev_campaign_contacts, and prev_outcome, replace with NaN
#    if there is at least one missing value

# Return rows with at least one missing value
missing_rows = df[df[['prev_days', 'n_prev_campaign_contacts', 'prev_outcome']]\
                  .isna().any(axis=1)]

# turn all 3 variables in the missing rows to -1 and NaN
df.loc[missing_rows.index, ['prev_days', 'n_prev_campaign_contacts']] = -1
df.loc[missing_rows.index, ['prev_outcome']] = np.nan

# 4. # Map prev_outcome categories and let na values be not_contacted
prev_outcome_dict = {
    'failure': 'failure',
    'success': 'success',
    'other': 'other',
    np.nan: 'not_contacted'
}
def map_prev_outcome(value):
    return prev_outcome_dict[value]

# Apply mapping to prev_outcome
df['prev_outcome'] = df['prev_outcome'].apply(map_prev_outcome)

#####
return df
```

```
In [6]: # Apply prepare_data function to both training and test data
train_df = prepare_data(train_df)
test_df = prepare_data(test_df)
```

C:\Users\gordon\AppData\Local\Temp\ipykernel_16772\3732427580.py:159: UserWarning: Parsing dates in %d/%m/%Y format when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specify a format to silence this warning.

```
df['last_contact_date'] = pd.to_datetime(df['last_contact_date'],
```

Impute Missing Values

```
In [7]: # Create X and Y dataset for both training and test
y_train = train_df['subscribed']
X_train = train_df.drop(columns=['subscribed'])

X_test = test_df
```

```
In [8]: # Define impute missing values transformer

impute_missing_values = ColumnTransformer(
    # ColumnTransformer that imputes missing values for variables with missing values
    [
        # iterative_imputer: use MICE to impute numeric variables
        ('iterative_imputer',
         IterativeImputer(random_state=1),
         ['age', 'avg_balance']),

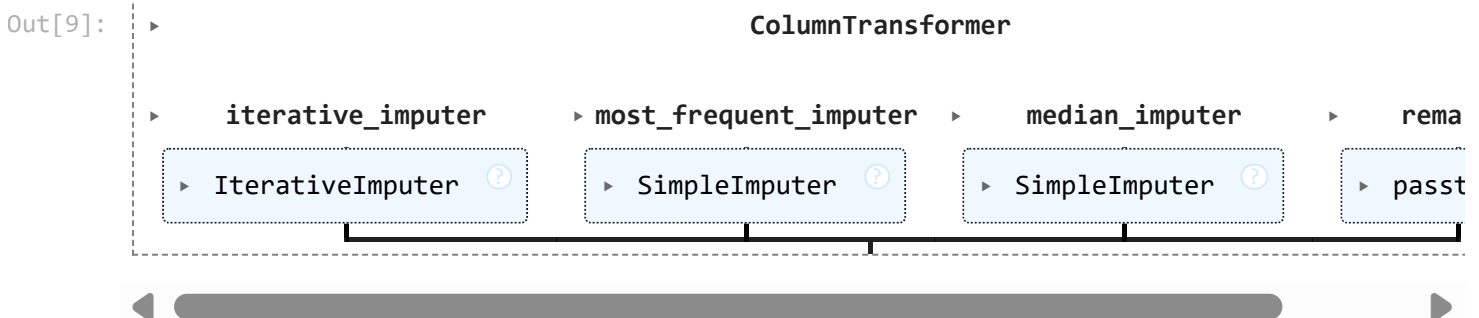
        # most_frequent_imputer: impute variables using their most frequent values
        ('most_frequent_imputer',
         SimpleImputer(strategy='most_frequent'),
         ['education', 'last_contact_weekday', 'last_contact_month',
          'last_contact_day', 'has_pers_loan', 'default', 'has_mortgage']),

        # median_imputer: impute variables with their median values
        ('median_imputer',
         SimpleImputer(strategy='median'),
         ['total_call_time', 'n_curr_campaign_contacts'])
    ],
    # remainder='passthrough': Leave all other variables (without missing values) unchanged
    remainder='passthrough'
)
```

```
In [9]: # Fit impute_missing_values to X_train
impute_missing_values.fit(X_train)
```

c:\Users\gordon\anaconda3\envs\aad2\Lib\site-packages\sklearn\compose_column_transformer.py:1667: FutureWarning:
The format of the columns of the 'remainder' transformer in ColumnTransformer.transformers_ will change in version 1.7 to match the format of the other transformers.
At the moment the remainder columns are stored as indices (of type int). With the same ColumnTransformer configuration, in the future they will be stored as column names (of type str).
To use the new behavior now and suppress this warning, use ColumnTransformer(force_int_remainder_cols=False).

```
warnings.warn(
```



```
In [10]: # Define function to apply the imputer fit

def apply_imputer_fit(transformer_imputer, X_df):
    """
    Apply the transformer imputer to specified X dataset to impute missing values.

    Parameters
```

```

-----
transformer_imputer: ColumnTransformer
    The fitted ColumnTransform imputer on X_train.
X_df: pandas.DataFrame
    Dataframe with feature variables to impute.

Returns
-----
X_df_imputed: pandas.DataFrame
    X dataframe with imputed missing values and original column names.
"""
# Apply transformer imputer
X_imputed = transformer_imputer.transform(X_df)
X_df_imputed = pd.DataFrame(X_imputed,
                             columns=transformer_imputer.get_feature_names_out())
X_df_imputed.columns = X_df_imputed.columns.str.replace(r'.*__', '', regex=True)

# Convert columns to numeric variables, and keep as float for categorical variables
for col in X_df_imputed.columns:
    try:
        X_df_imputed[col] = pd.to_numeric(X_df_imputed[col])
    except (ValueError, TypeError):
        pass

return X_df_imputed

```

```

In [11]: # Apply impute_missing values fit to X_train and Y_train
X_train_imputed = apply_imputer_fit(impute_missing_values, X_train)
X_test_imputed = apply_imputer_fit(impute_missing_values, X_test)

```

2. Feature Engineering

Create Ratio Variables

```

In [12]: # Define function to create ratio variables
def create_ratio_variables(X_df):
    """
    Create a new variable, average call time, which is the ratio of the
    variables 'total_call_time' and 'n_curr_campaign_contacts'.

    Parameters
    -----
    X_df: pandas.DataFrame
        Imputed dataframe with feature variables.

    Returns
    -----
    X_df: pandas.DataFrame
        Imputed dataframe with an additional 'avg_call_time' feature variable.
    """
    # Calculate the average call time and create 'avg_call_time' variable
    X_df['avg_call_time'] = X_df['total_call_time']/X_df['n_curr_campaign_contacts']

    return X_df

```

```

In [13]: # Apply ratio_variable to dataset
X_train_engineered = create_ratio_variables(X_train_imputed)
X_test_engineered = create_ratio_variables(X_test_imputed)

```

Create Cyclical Features

In [14]: *# Define function to add cyclic relationships*

```
def add_cyclic_relationships(X_df):
    """
    Allow for cyclic relationships for 'last_contact_month', 'last_contact_day',
    and 'last_contact_weekday'.

    Parameters
    -----
    X_df: pandas.DataFrame
        Imputed dataframe with feature variables.

    Returns
    -----
    X_df: pandas.DataFrame
        Imputed dataframe with feature variables 'last_contact_month', 'last_contact_day',
        and 'last_contact_weekday', replaced by 'month_sin', 'month_cos', 'day_sin',
        'day_cos', 'weekday_sin', and 'weekday_cos'.
    """
    # Month (1-12)
    X_df['month_sin'] = np.sin(2 * np.pi * X_df['last_contact_month'] / 12)
    X_df['month_cos'] = np.cos(2 * np.pi * X_df['last_contact_month'] / 12)

    # Day (1-31)
    X_df['day_sin'] = np.sin(2 * np.pi * X_df['last_contact_day'] / 31)
    X_df['day_cos'] = np.cos(2 * np.pi * X_df['last_contact_day'] / 31)

    # Weekday (0-6)
    X_df['weekday_sin'] = np.sin(2 * np.pi * X_df['last_contact_weekday'] / 7)
    X_df['weekday_cos'] = np.cos(2 * np.pi * X_df['last_contact_weekday'] / 7)

    # Optionally drop originals
    X_df = X_df.drop(columns=['last_contact_month', 'last_contact_day',
                             'last_contact_weekday'])

    # return df
    return X_df
```

In []: *# Apply add_cyclical_features function*

```
X_train_engineered = add_cyclic_relationships(X_train_engineered)
X_test_engineered = add_cyclic_relationships(X_test_engineered)
```

One-Hot Encoding and Scaling

In [16]: *# Create column transformer to one-hot encode categorical variables
and scale numeric variables*

```
def log(x):
    """Natural log transform with 1 offset to allow 0 values."""
    return np.log1p(x)

def shifted_log(x):
    """Natural log transform after offsetting by 9000 for negative values"""
    offset = 9000
    # Exception test: negative of the offset must be greater than the smallest value of x
    if np.nanmin(x) < -offset:
        raise Exception(
            f"Offset of {offset} is too small to offset the minimum value {np.nanmin(x):.3f}"
```

```

    )
    return np.log1p(offset + x)

log_scale_transformer = make_pipeline(
    FunctionTransformer(func = log, feature_names_out = 'one-to-one'),
    MinMaxScaler()
)

shifted_log_scale_transformer = make_pipeline(
    FunctionTransformer(func = shifted_log, feature_names_out = 'one-to-one'),
    MinMaxScaler()
)

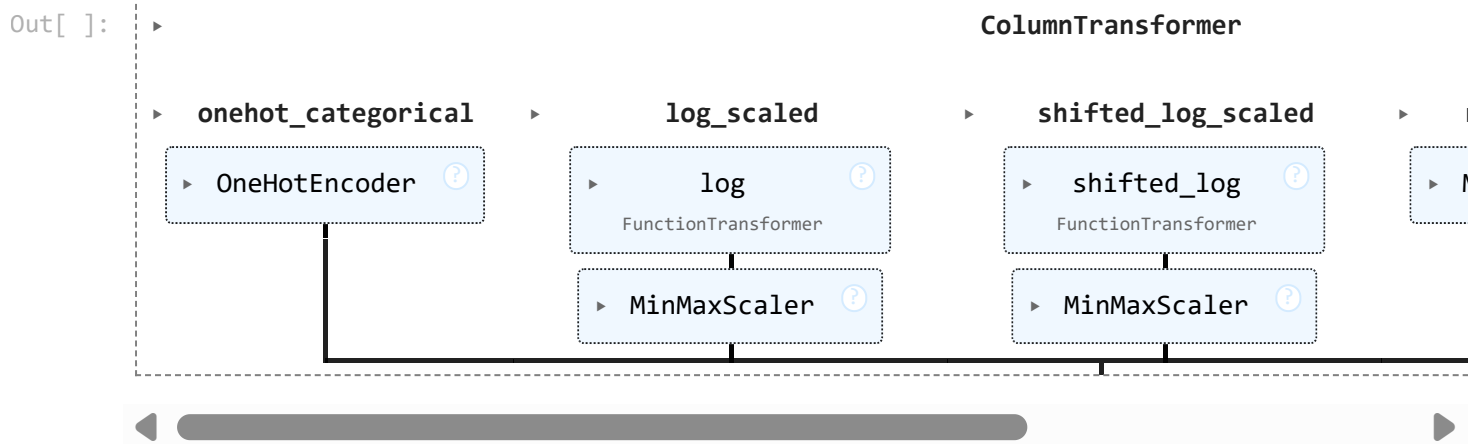
column_transformer = ColumnTransformer(
    [
        # One-hot encoding categorical features
        (
            'onehot_categorical',
            OneHotEncoder(drop = 'first'),
            ['occupation', 'marital_status', 'contact_type', 'prev_outcome'],
        ),
        # Log transform and min-max scaling to numeric features
        (
            'log_scaled',
            log_scale_transformer,
            ['total_call_time', 'n_curr_campaign_contacts', 'avg_call_time'],
        ),
        # Shifted log transform and min-max scaling to 'avg_balance',
        # which has negative values
        (
            'shifted_log_scaled',
            shifted_log_scale_transformer,
            ['avg_balance'],
        ),
        # Min-max scaling to numeric variables with minimal skew
        (
            'min_max_scaler',
            MinMaxScaler(),
            ['age', 'education', 'prev_days', 'n_prev_campaign_contacts'],
        ),
        # Leave remaining numeric variables unchanged
        (
            'passthrough_numeric',
            'passthrough',
            ['has_mortgage', 'has_pers_loan', 'default',
             'month_sin', 'month_cos', 'day_sin', 'day_cos', 'weekday_sin', 'weekday_cos'],
        ),
    ],
    remainder = 'drop',
)

```

```

In [ ]: # Fit column_transformer to X_train_engineered
column_transformer.fit(X_train_engineered)

```



```
In [18]: # Apply column_transformer fit to X_train_engineered
transformed_df = column_transformer.transform(X_train_engineered)
X_train_engineered = pd.DataFrame(transformed_df,
                                   columns=column_transformer.get_feature_names_out())
```

```
In [19]: # Apply column_transformer fit to X_test_engineered
transformed_df = column_transformer.transform(X_test_engineered)
X_test_engineered = pd.DataFrame(transformed_df,
                                   columns=column_transformer.get_feature_names_out())
```

Feature Selection: Remove Highly Correlated Features

```
In [20]: # Create function to drop correlated features
def drop_correlated_features(X_df):
    """
    Drop certain variables that are highly correlated, namely 'log_scaled__total_call_time',
    and 'onehot_categorical__prev_outcome_not_contacted'.

    Parameters
    -----
    X_df: pandas.DataFrame
        Imputed and feature engineered dataframe with feature variables.

    Returns
    -----
    X_df: pandas.DataFrame
        Imputed and feature engineered dataframe without variables
        'log_scaled__total_call_time', and
        'onehot_categorical__prev_outcome_not_contacted'.
    """
    # Drop features
    X_df = X_df.drop(columns=['log_scaled__total_call_time',
                              'onehot_categorical__prev_outcome_not_contacted'])

    return X_df
```

```
In [21]: # Apply drop features function
X_train_engineered = drop_correlated_features(X_train_engineered)
X_test_engineered = drop_correlated_features(X_test_engineered)
```

3. Modelling

```
In [22]: # Define model evaluation function
def evaluate_model(model, X_train, y_train, cv=10):
    """
```

Evaluate a classification model's performance using cross validation with F1 scoring.

Parameters

model: estimator object

A classification model from sklearn, such as LogisticRegression and DecisionTreeClassifier.

X_train: pandas.DataFrame

An imputed and feature engineered dataframe with all selected feature variables.

y_train: pandas.DataFrame

A dataframe containing the target label corresponding to X_train

cv: int, default = 10

Number of cross-validation folds; the integer must be greater than 1.

Returns

None

Prints the mean and standard deviation of the training and validation F1 over all cross-validation folds.

"""

Fit models

```
model_scores = cross_validate(model, X_train, y_train,
                              scoring='f1', cv=cv,
                              return_train_score=True)
```

Evaluate model

```
train_f1_mean = np.mean(model_scores['train_score'])
val_f1_mean = np.mean(model_scores['test_score'])
train_f1_std = np.std(model_scores['train_score'])
val_f1_std = np.std(model_scores['test_score'])
```

```
print(f'Training F1: {train_f1_mean:.3f} +/- {train_f1_std:.4f}')
```

```
print(f'Validation F1: {val_f1_mean:.3f} +/- {val_f1_std:.4f}')
```

In [23]: **def** optimal_threshold(model, X_train, y_train, cv=10, n_repeats=30, random_state=1):

"""

Find the optimal threshold that maximizes F1 for each n repeated Stratified K-fold cross validation.

Parameters

model: estimator object

A classification model from sklearn, such as LogisticRegression and DecisionTreeClassifier.

X_train: pandas.DataFrame

An imputed and feature engineered dataframe with all selected feature variables.

y_train: pandas.DataFrame

A dataframe containing the target label corresponding to X_train

cv: int, default = 10

Number of cross-validation folds; the integer must be greater than 1.

n_repeats: int, default = 30

Number of times to repeat the cross validation procedure with different shuffles.

random_state: int, default = 1

Seed to ensure reproducibility of the threshold and F1 results.

Returns

None

Print the mean and standard deviation of the optimal threshold and cross validation F1 score over all n repeated stratified K-fold.

"""

Store best threshold value for each n_repeat and its respective F1 score

best_thresholds = []

f1_scores_at_best = []

for i **in** range(n_repeats):

Shuffle data differently for each repeat

skf = StratifiedKFold(n_splits=cv, shuffle=True, random_state=random_state+i)

Out-of-fold predicted probabilities

y_train_proba = cross_val_predict(model, X_train, y_train,
cv=skf,
method='predict_proba')[:, 1]

Precision-recall curve

precision, recall, thresholds = precision_recall_curve(y_train, y_train_proba)

Compute F1, added 1e-10 to prevent divided by 0

f1_scores = (2 * precision * recall) / (precision + recall + 1e-10)

Select best threshold

best_idx = np.argmax(f1_scores)

best_threshold = thresholds[best_idx]

best_f1 = f1_scores[best_idx]

best_thresholds.append(best_threshold)

f1_scores_at_best.append(best_f1)

Calculate the mean and standard deviation of the threshold and its respective F1 score

mean_threshold = np.mean(best_thresholds)

std_threshold = np.std(best_thresholds)

mean_f1 = np.mean(f1_scores_at_best)

std_f1 = np.std(f1_scores_at_best)

Print results

print(f"Mean optimal threshold: {mean_threshold:.3f} +/- {std_threshold:.4f}")

print(f"Mean CV F1 at optimal threshold: {mean_f1:.4f} +/- {std_f1:.4f}")

Selected Best Model: Tuned GBM

In [24]: *# Specify tuned GBM model's hyperparameters*

```
tuned_gbm = HistGradientBoostingClassifier(max_depth=14,  
                                           max_leaf_nodes=45,  
                                           min_samples_leaf=300,  
                                           max_iter=250,  
                                           learning_rate=0.15,  
                                           l2_regularization=20,  
                                           random_state=1)
```

Evaluate tuned GBM model via 10-fold CV using the evaluate_model function

evaluate_model(tuned_gbm, X_train_engineered, y_train, cv=10)

Training F1: 0.734 +/- 0.0086

Validation F1: 0.685 +/- 0.0107

In [585...

Find the mean optimal threshold and mean CV F1 using the optimal_threshold function

optimal_threshold(tuned_gbm, X_train_engineered, y_train, random_state=1)

Mean optimal threshold: 0.342 +/- 0.0168
Mean CV F1 at optimal threshold: 0.7057 +/- 0.0013

4. Apply Model on Full Training and Test Dataset

```
In [ ]: # Fit the tuned GBM model to full training dataset
tuned_gbm.fit(X_train_engineered, y_train)
```

```
Out[ ]: ▾ HistGradientBoostingClassifier ⓘ ?

HistGradientBoostingClassifier(l2_regularization=20, learning_rate=0.15,
                               max_depth=14, max_iter=250, max_leaf_nodes=45,
                               min_samples_leaf=300, random_state=1)
```

```
In [594... # Obtain test data's prediction probabilities
y_test_pred_proba = tuned_gbm.predict_proba(X_test_engineered)[:, 1]
```

```
In [595... # Apply mean threshold to the test data's prediction probabilities
threshold = 0.342
y_test_pred_f1 = np.copy(y_test_pred_proba)
y_test_pred_f1[y_test_pred_f1 >= threshold] = 1
y_test_pred_f1[y_test_pred_f1 < threshold] = 0
```

```
In [596... # Create dataframe with test data ID and predictions
predict_df = pd.DataFrame({
    'ID': test_ids,
    'prediction': y_test_pred_f1
})
```

```
In [597... # Export final test data predictions to .csv file
predict_df.to_csv('Submission-1315187.csv', index=False)
```